Java-based FODA: Architecture & Development

(Fuzzy-Ontology-based Diagnostics Architecture)

Roadmap

This document describes the Java-targeted architecture, component design, ontology sketch, data and control flows, Python-based ML integration, implementation notes, and a step-by-step roadmap for a proof-of-concept prototype of:

Development of a Fuzzy-Ontology-Based Method for Explainable Diagnostics of Distributed Systems

1. High-level overview

Goal:
Build a **Java + Python hybrid prototype** that demonstrates **explainable diagnostics** in a distributed microservice environment using **fuzzy logic, ontology reasoning, and ML-based anomaly detection**.

The system should:

- Detect anomalies and uncertain behavior using ML models (Python side).
- Represent diagnostic knowledge semantically via OWL ontology (Java side).
- Produce **human-readable explanations** enriched with fuzzy confidence values.
- Allow operators to trace the reasoning chain from raw data to ontology-based reasoning.

Use case (recommended):
Microservice system where each service reports performance metrics (CPU, latency, throughput).
A **Python ML model** detects anomalies → sends results to the **Fuzzy Diagnostic Engine**

(Java) → fuzzy-ontology layer generates structured explanations stored in RDF for visualization and analysis.

2. Core components (Java–Python hybrid)

1. Monitored Services (A, B, C)

- Implemented as **Spring Boot 3.x microservices**.
- Each exposes /metrics, /status, /diagnose.
- Publishes metric data to the message bus (Kafka).

2. ML Anomaly Detection Microservice (Python-based)

- Python FastAPI or Flask microservice (ml-anomaly-detector).
- Trains and serves anomaly detection models (Isolation Forest, Autoencoder, or LSTM).
- Uses libraries: **scikit-learn**, **TensorFlow**, or **PyOD**.
- Exposes endpoints:
  - /predict — returns anomaly probability.
  - /explain — returns feature importance using **SHAP** or **LIME**.
- Sends results to the **Fuzzy Diagnostic Engine** via Kafka topic ml-output.

3. Fuzzy Diagnostic Engine (FDE) — Java

- Spring Boot service performing **fuzzy reasoning and ontology mapping**.
- Consumes ML outputs and metric data.
- Uses **Drools Fuzzy** or **jFuzzyLogic** to compute confidence levels.
- Converts quantitative inputs to linguistic terms ("high CPU", "moderate latency").
- Produces **Diagnostic Explanation Payloads** (see schema below).

4. Ontology Mapper

- Java module linking fuzzy outputs to ontology classes.
- Uses **Apache Jena + OWL API** for semantic reasoning.
- Establishes relationships: hasCause, affectsService, suggestsAction.

5. Explainability Store (E-Store)

- RDF TDB or Fuseki triple store managed by Jena.
- Stores diagnostic explanations and causal graphs.
- Exposes SPARQL endpoint (/sparql, /queryExplanation).

6. Explanation API (E-Service)

- Spring Boot REST microservice for explanation management.
- Endpoints:
    - /submitExplanation — receives JSON from FDE.
    - /getExplanation?id=UUID — returns ontology-enriched explanation.
    - /getConfidenceSummary — computes average Fuzzy Confidence Index (FCI).

7. Operator Dashboard

- Spring Boot + Thymeleaf or React frontend.
- Displays:
    - Service health and anomaly trends.
    - Fuzzy confidence charts and ontology graphs.
    - Human-readable explanation chains.

8. Message Bus / Eventing

- **Apache Kafka** connects Java and Python layers.
- Topics:
  - metrics (from monitored services),
  - ml-output (from Python ML),
  - fuzzy-explanations (from FDE).

3. Diagnostic Explanation Payload (JSON schema)


```
{
 "explanationId": "uuid",
 "timestamp": "ISO8601",
 "serviceId": "service-A",
 "diagnosticResult": "ResourceContention",
 "fuzzyConfidence": 0.87,
 "mlPrediction": {"anomalyScore": 0.92, "model": "isolation_forest_v1"},
 "importantFeatures": [
  {"name": "cpu_util", "importance": 0.45},
  {"name": "latency", "importance": 0.37}
 ],
 "fuzzyInputs": [
  {"metric": "cpu_util", "value": 0.93, "membership": "high"},
  {"metric": "latency", "value": 320, "membership": "moderate"}
 ],
 "ontologyConcepts": [
  {"iri": "http://example.org/ontology#ResourceContention",
   "label": "Resource Contention",
   "confidence": 0.9}
 ],
 "causalChain": [
  {"cause": "HighCPU", "effect": "ServiceDegradation", "confidence": 0.82}
 ],
 "suggestedActions": [
  {"action": "scale", "target": "service-B", "priority": 0.75}
 ],
```

"provenance": {"engine": "FDE-v2", "mlSource": "PythonFastAPI"}
}

4. Data / Control Flow

1. Services A/B/C → publish metrics → Kafka metrics topic.
2. Python ML service → consumes metrics → returns anomaly predictions → Kafka ml-output.
3. Java FDE → consumes ml-output → applies fuzzy logic → generates explanation JSON.
4. Ontology Mapper → enriches explanation → stores in **E-Store**.
5. E-Service → exposes APIs for visualization.
6. Operator Dashboard → queries RDF store and displays explanations.

5. Ontology sketch (core classes & properties)

Classes:

- DiagnosticEvent
- Metric
- Service
- AnomalyType
- MLModel
- Action

Properties:

- hasCause (DiagnosticEvent → AnomalyType)
- detectedBy (DiagnosticEvent → MLModel)
- affectsService (AnomalyType → Service)
- hasConfidence (DiagnosticEvent → float)
- suggestsAction (AnomalyType → Action)

Example subclasses:

- HighCPU, NetworkPartition, ResourceContention, ModelDrift, LatencySpike.

Create ontology in **Protégé**, export .owl, and load into **Jena TDB**.

6. Implementation details & recommended libraries

Category	Recommended Tools
Java version	OpenJDK 17 / 21
Framework	Spring Boot 3.x (WebFlux or WebMVC)
Messaging	Apache Kafka (JSON payloads)
Ontology	Apache Jena (Fuseki, TDB) + OWL API
Fuzzy Logic	Drools Fuzzy / jFuzzyLogic / Fuzzy4J
Python ML stack	scikit-learn, PyOD, TensorFlow, SHAP, LIME, FastAPI
Database	PostgreSQL (metrics), RDF store (explanations)
Containerization	Docker + Docker Compose
Observability	OpenTelemetry + Prometheus
UI	React or Spring Thymeleaf dashboard

7. Development roadmap (milestones + estimated durations)

Phase	Duration	Description

Phase 0 — Setup & Preparation (1 week)

- Initialize Git repo, base Java + Python services, Docker setup, Protégé ontology skeleton.

Phase 1 — Basic Microservice Prototype (2 weeks)

- Implement Services A/B/C and Kafka messaging.
- Verify metrics publishing and collection.

Phase 2 — Python ML Anomaly Detector (3 weeks)

- Build FastAPI service with scikit-learn Isolation Forest or Autoencoder.
- Implement /predict and /explain endpoints with SHAP explanations.
- Publish predictions to Kafka (ml-output).

Phase 3 — Fuzzy Diagnostic Engine (3 weeks)

- Consume ML output → apply fuzzy inference (Drools/jFuzzyLogic).
- Generate explanation payloads with fuzzy confidence.

Phase 4 — Ontology Integration (3 weeks)

- Develop OWL ontology; link fuzzy + ML outputs to semantic entities.
- Store enriched explanations in Jena TDB.

Phase 5 — Explanation API & Dashboard (2 weeks)

- Build E-Service (REST) and visualization dashboard.
- Show explanations, causal chains, and confidence levels.

Phase 6 — Evaluation & Testing (3 weeks)

- Fault injection: simulate CPU spikes, latency, network issues.
- Measure diagnostic accuracy, FCI, EDAR, and interpretability.

🕒 **Total PoC time: ≈ 16 weeks (4 months)**

8. Test plan / Evaluation strategy

1. **Functional tests:** Validate ML predictions, fuzzy inference, ontology mapping.
2. **Integration tests:** Verify message flow between Python and Java layers.
3. **Performance tests:** Measure latency from metric ingestion to explanation output.
4. **Effectiveness:** Compare FODIM (Fuzzy-Ontology Diagnostic Inference Method) vs pure ML baseline.
5. **User evaluation (optional):** Operator trust and explanation clarity rating.

Metrics:

- Diagnostic Accuracy (DA)

- Fuzzy Confidence Index (FCI)
- Explanation–Data Agreement Rate (EDAR)
- System Overhead (SO)
- Operator Trust Score (OTS)

9. Security & Provenance Considerations

- Use HTTPS and Kafka SASL/SSL between Java and Python.
- Sign JSON payloads for authenticity.
- Sanitize ML feature outputs; avoid raw sensitive data.
- Store provenance info (mlModel, version, explainMethod) in RDF.

10. Minimal MVP Checklist

✅ Java services (A/B/C) + Kafka setup
✅ Python FastAPI ML microservice (Isolation Forest + SHAP)
✅ FDE consuming ml-output → fuzzy reasoning
✅ Ontology model in Protégé + loaded into Jena TDB
✅ E-Service REST endpoints
✅ Basic dashboard visualization

11. Example Fuzzy Rule (pseudocode)

```
rule "High CPU + ML anomaly → Resource Contention"
when
  MLPrediction(anomalyScore > 0.8)
  Metric(name == "cpu_util", value > 0.85)
then
  insert(new DiagnosticEvent("ResourceContention", confidence = 0.88));
end
```

12. Appendix — Hybrid ML Integration Notes

If advanced explainers are needed:

- Use **SHAP** or **Integrated Gradients** in Python ML service.
- FDE receives a simplified version of the SHAP output (feature → importance).
- Jena Ontology maps these features to semantic concepts (e.g., cpu_util → ResourceUsage).
- Explanation stored as a combined fuzzy-ML-ontology graph.